UNIVERSITÉ ABDELMALEK ESSAADI
Ecole Nationale des Sciences Appliquées d'Al Hoceima
Département Mathématique et Informatique
Module : J2EE / Java EE
Filière : TDIA 2

RAPPORT DE TP – JAVA EE (MVC1)

Application de gestion des produits avec Servlets, Filters, JSP et JSTL

TP N°4

Encadré par : Pr. Mohamed Cherradi
Réalisé par : Essouiri Aya

Année universitaire : 2025 / 2026

Table des matières

1 Introduction **2**

2 Objectifs du projet **2**

3 Architecture du système **2**

4 Implémentation du système **4**

4.1 Interfaces utilisateur 4
4.2 Classe Produit 6
4.3 Couche DAO 7
4.3.1 Interface ProduitDAO 7
4.3.2 Implémentation ProduitImpl 8
4.4 Couche Service 8
4.5 Couche Web (Servlets) 10
4.5.1 Ajout de produit 10
4.5.2 Affichage des produits 10
4.5.3 Modification d'un produit 11
4.5.4 Mise à jour d'un produit 11
4.5.5 Suppression d'un produit 12
4.6 Interface principale 12
4.7 Implémentation de l'authentification 13
4.8 Implémentation des filtres (Filters) 14
4.8.1 AuthFilter 14
4.9 Configuration du fichier web.xml 15
4.10 Gestion des erreurs 17

5 Mécanisme d'authentification **17**

6 Gestion des rôles **17**

7 Utilisation des Filters **18**

7.1 AuthFilter 18
7.2 RoleFilter 18

8 Gestion des erreurs **18**

9 Fonctionnement global **18**

10 Conclusion **19**

11 Perspectives d'amélioration **19**

1 Introduction

Dans le cadre du module de développement des applications web en Java EE, ce travail pratique a pour objectif la conception et la réalisation d'une application web dynamique dédiée à la gestion des produits. Cette application repose sur les technologies fondamentales de la plateforme Java EE, notamment les Servlets, les JSP (Java Server Pages) ainsi que la bibliothèque JSTL.

L'architecture adoptée est de type MVC1 combinée à une structuration en trois couches (3-tier architecture), permettant une séparation claire des responsabilités entre la présentation, la logique métier et l'accès aux données.

Dans une optique d'amélioration de la robustesse et de la sécurité de l'application, plusieurs mécanismes avancés ont été intégrés, tels que l'authentification des utilisateurs, la gestion des rôles (autorisation), la gestion centralisée des erreurs ainsi que l'utilisation des filtres (Filters) pour intercepter et contrôler les requêtes HTTP.

2 Objectifs du projet

Ce projet vise à atteindre plusieurs objectifs pédagogiques et techniques :

— Comprendre et appliquer l'architecture MVC1 dans une application web Java EE
— Mettre en œuvre une architecture en couches (DAO, Service, Web)
— Développer des fonctionnalités CRUD complètes
— Implémenter un mécanisme d'authentification sécurisé
— Gérer les droits d'accès via un système de rôles
— Utiliser les filtres pour sécuriser les accès aux ressources
— Mettre en place une gestion efficace des erreurs

3 Architecture du système

L'application est basée sur une architecture multicouche permettant de garantir la modularité, la maintenabilité et l'évolutivité du système.

$$\text{Client} \rightarrow \text{Filter} \rightarrow \text{Servlet} \rightarrow \text{Service} \rightarrow \text{DAO} \rightarrow \text{Source de données}$$

Chaque couche possède un rôle bien défini :

— **Client** : interface utilisateur (navigateur web)
— **Filter** : interception des requêtes pour la sécurité (authentification, autorisation)
— **Servlet** : contrôleur assurant le traitement des requêtes HTTP
— **Service** : implémentation de la logique métier
— **DAO** : gestion de l'accès aux données
— **Source de données** : stockage (mémoire ou base de données)

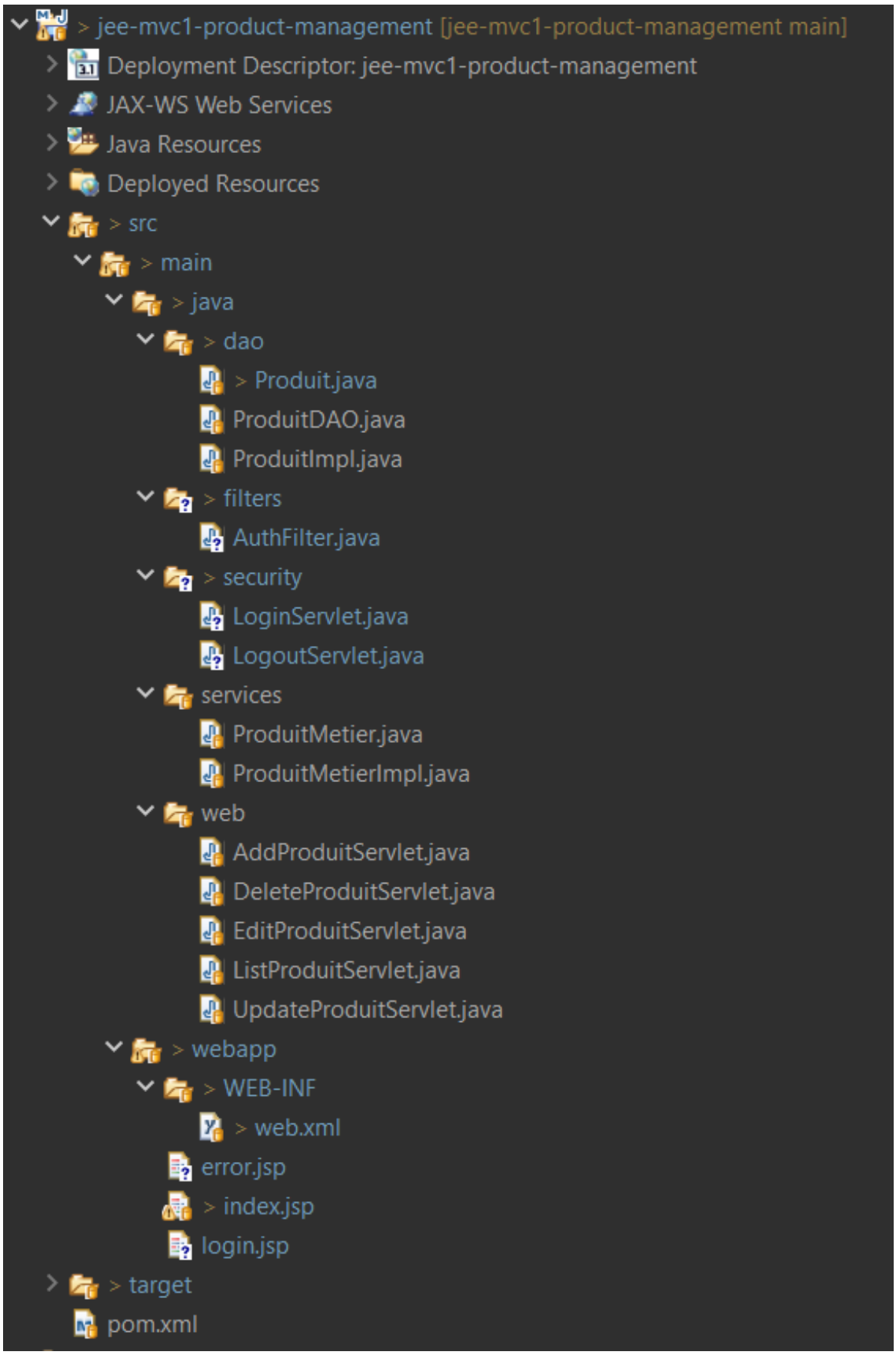


FIGURE 1 – Architecture multicouche du système

4 Implémentation du système

Cette section présente les différentes composantes de l'application à travers des captures d'écran des interfaces ainsi que des extraits du code source.

4.1 Interfaces utilisateur

Les interfaces ont été développées en JSP afin de permettre une interaction fluide avec l'utilisateur.

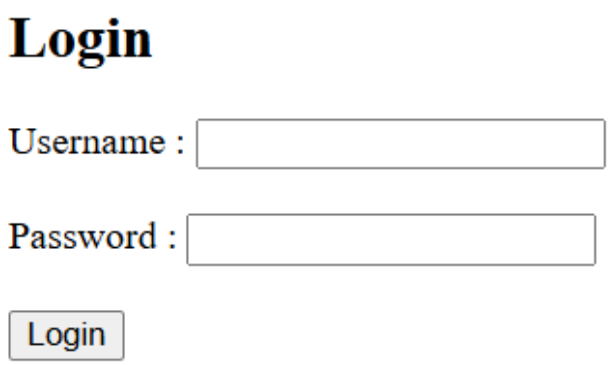
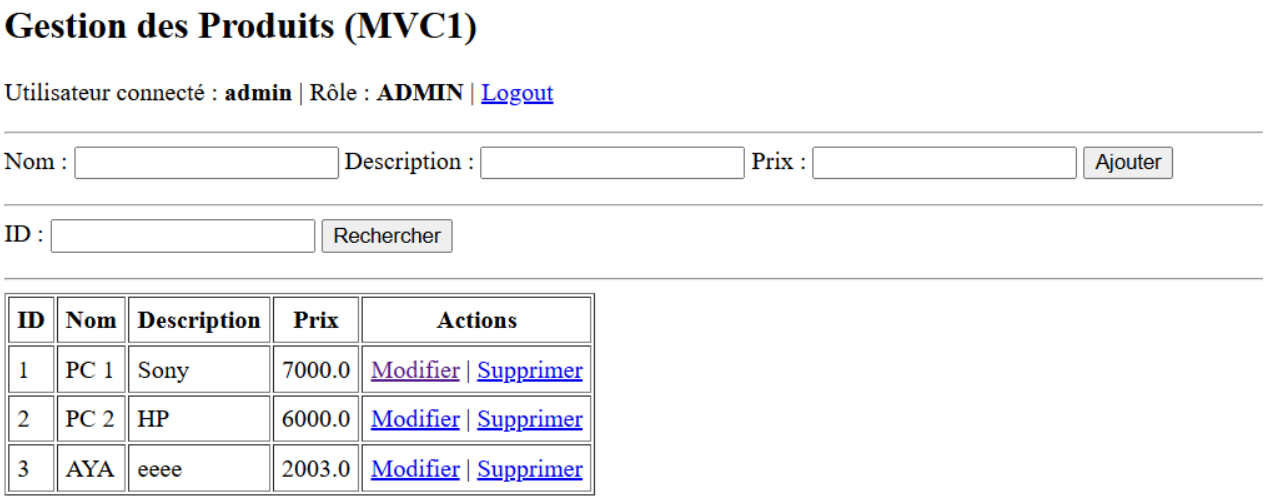


FIGURE 2 – Page de connexion



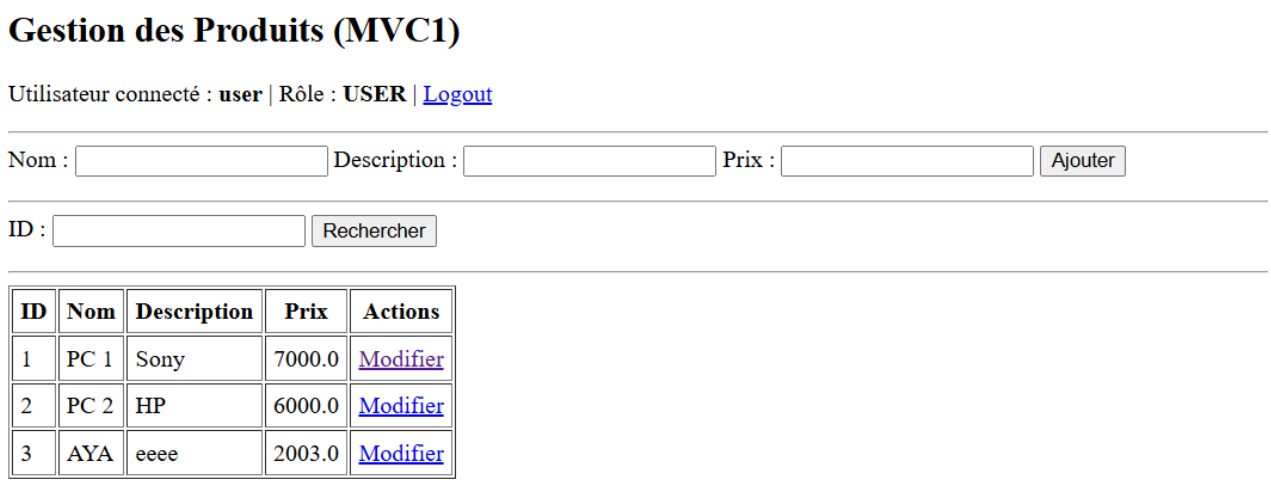


FIGURE 3 – Interface principale affichant la liste des produits avec un formulaire d'ajout, accessible aux utilisateurs selon leur rôle (User ou Admin)

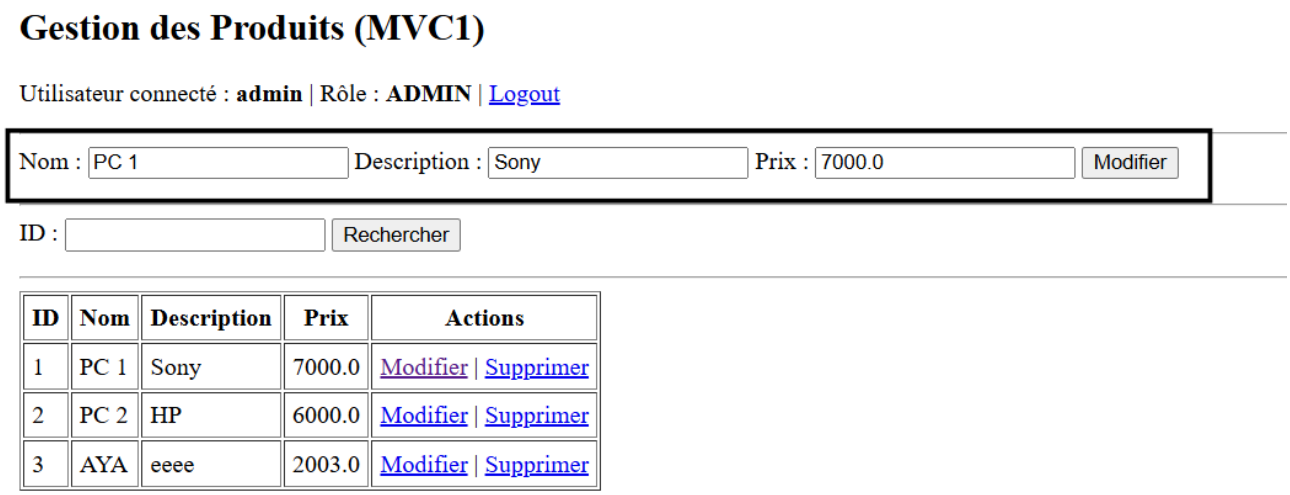


FIGURE 4 – Formulaire de modification d'un produit

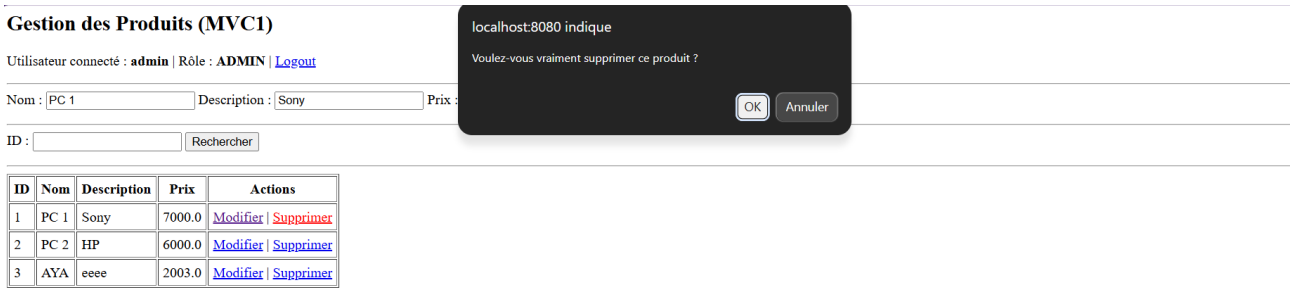


FIGURE 5 – Interface de suppression d'un produit

4.2 Classe Produit

La classe *Produit* représente l'entité principale du système. Elle modélise les informations essentielles relatives à un produit, notamment son identifiant (*idProduit*), son nom (*nomProduit*), sa description (*description*) ainsi que son prix (*prix*). Cette classe est utilisée dans l'ensemble de l'application, notamment par la couche DAO pour la gestion des données et par la couche Service pour l'application de la logique métier.

```java
package dao;

public class Produit{
    private long idProduit;
    private String nom;
    private String description;
    private Double prix;

    public Produit() {}

    public Produit(String nom, String description, Double prix) {
        this.setNom(nom);
        this.setDescription(description);
        this.setPrix(prix);
    }

    public long getIdProduit() { return idProduit; }
    public void setIdProduit(long idProduit) { this.idProduit = idProduit; }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public Double getPrix() { return prix; }
    public void setPrix(Double prix) { this.prix = prix; }
}
```

FIGURE 6 – Classe Produit

4.3 Couche DAO

4.3.1 Interface ProduitDAO

```java
package dao;

import java.util.List;

public interface ProduitDAO {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
    void updateProduit(Produit p);
}
```

FIGURE 7 – Interface ProduitDAO

4.3.2 Implémentation ProduitImpl

```java
public class ProduitImpl implements ProduitDAO {

    private List<Produit> produits = new ArrayList<>();

    public void init(){
        addProduit(new Produit("PC 1","Sony",7000.0));
        addProduit(new Produit("PC 2","HP",6000.0));
    }

    public void addProduit(Produit p) {
        p.setIdProduit((long) (produits.size()+1));
        produits.add(p);
    }

    public void deleteProduit(Long id) {
        produits.remove(getProduitById(id));
    }

    public Produit getProduitById(Long id) {
        for(Produit p:produits){
            if(p.getIdProduit()== id) return p;
        }
        return null;
    }

    public List<Produit> getAllProduits() {
        return produits;
    }

    public void updateProduit(Produit p) {
        for(Produit pr:produits){
            if(pr.getIdProduit() == p.getIdProduit()){
                pr.setNom(p.getNom());
                pr.setDescription(p.getDescription());
                pr.setPrix(p.getPrix());
            }
        }
    }
}
```

FIGURE 8 – Implémentation ProduitImpl

4.4 Couche Service

La couche service assure la logique métier de l'application.

```java
public interface ProduitMetier {
    void addProduit(Produit p);
    void deleteProduit(Long id);
    List<Produit> getAllProduits();
    Produit getProduitById(Long id);
    void updateProduit(Produit p);
}
```

FIGURE 9 – Interface ProduitMetier

```java
public class ProduitMetierImpl implements ProduitMetier {

    private static ProduitMetierImpl instance;
    private ProduitDAO dao;

    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init();
    }

    public static ProduitMetierImpl getInstance() {
        if(instance == null) instance = new ProduitMetierImpl();
        return instance;
    }

    public void addProduit(Produit p){
        dao.addProduit(p);
    }
    public void deleteProduit(Long id){
        dao.deleteProduit(id);
    }
    public List<Produit> getAllProduits(){
        return dao.getAllProduits();
    }
    public Produit getProduitById(Long id){
        return dao.getProduitById(id);
    }
    public void updateProduit(Produit p){
        dao.updateProduit(p);
    }
}
```

FIGURE 10 – Implémentation ProduitMetierImpl

4.5 Couche Web (Servlets)

4.5.1 Ajout de produit

```java
public class AddProduitServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    List<Produit> produits = new ArrayList<Produit>();

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {

        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        Double prix = Double.parseDouble(req.getParameter("prix"));

        Produit p = new Produit(nom, description, prix);
        metier.addProduit(p);
        produits.add(p);

        req.setAttribute("listeProduits", metier.getAllProduits());
        req.getRequestDispatcher("index.jsp").forward(req, resp);
    }
}
```

FIGURE 11 – AddProduitServlet

4.5.2 Affichage des produits

```java
public class ListProduitServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {

        String idStr = req.getParameter("idProduit");

        List<Produit> liste = new ArrayList<>();

        if (idStr != null && !idStr.isEmpty()) {
            try {
                Long id = Long.parseLong(idStr);

                Produit p = metier.getProduitById(id);

                if (p != null) {
                    liste.add(p);
                }

            } catch (NumberFormatException e) {
                liste = metier.getAllProduits();
            }
        } else {
            liste = metier.getAllProduits();
        }

        req.setAttribute("listeProduits", liste);
        req.getRequestDispatcher("index.jsp").forward(req, resp);
    }
}
```

FIGURE 12 – ListProduitServlet

4.5.3 Modification d'un produit

```java
public class EditProduitServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
           throws ServletException, IOException {

        Long id = Long.parseLong(req.getParameter("id"));

        Produit p = metier.getProduitById(id);

        req.setAttribute("produitEdit", p);
        req.setAttribute("listeProduits", metier.getAllProduits());

        req.getRequestDispatcher("index.jsp").forward(req, resp);
     }
}
```

FIGURE 13 – EditProduitServlet

4.5.4 Mise à jour d'un produit

```java
public class UpdateProduitServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        Long id = Long.parseLong(req.getParameter("idProduit"));
        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        Double prix = Double.parseDouble(req.getParameter("prix"));

        Produit p = new Produit();
        p.setIdProduit(id);
        p.setNom(nom);
        p.setDescription(description);
        p.setPrix(prix);

        metier.updateProduit(p);

        req.setAttribute("listeProduits", metier.getAllProduits());
        req.getRequestDispatcher("index.jsp").forward(req, resp);

    }
}
```

FIGURE 14 – UpdateProduitServlet

4.5.5 Suppression d'un produit

```java
public class DeleteProduitServlet extends HttpServlet{

    private static final long serialVersionUID = 1L;

    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {

        Long id = Long.parseLong(req.getParameter("id"));
        metier.deleteProduit(id);
        resp.sendRedirect("listProduits");
    }
}
```

FIGURE 15 – DeleteProduitServlet

4.6 Interface principale

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<html>
<head>
    <title>Gestion des Produits MVC1</title>
</head>

<body>

<h2>Gestion des Produits (MVC1)</h2>

<form action="${produitEdit != null ? 'updateProduit' : 'addProduit'}" method="post">
    <input type="hidden" name="idProduit" value="${produitEdit.idProduit}" />

    Nom: <input type="text" name="nom" value="${produitEdit.nom}" required />
    Description: <input type="text" name="description" value="${produitEdit.description}" required />
    Prix: <input type="text" name="prix" value="${produitEdit.prix}" required />

    <input type="submit" value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />
</form>

<hr/>

<form action="listProduits" method="get">
    ID: <input type="text" name="idProduit" />
    <input type="submit" value="Rechercher" />
</form>

<hr/>

<table border="1" cellpadding="5">
    <tr>
        <th>ID</th>
        <th>Nom</th>
        <th>Description</th>
        <th>Prix</th>
        <th>Actions</th>
    </tr>

    <c:forEach var="p" items="${listeProduits}">
        <tr>
            <td>${p.idProduit}</td>
            <td>${p.nom}</td>
            <td>${p.description}</td>
            <td>${p.prix}</td>
            <td>
                <a href="editProduit?id=${p.idProduit}">Modifier</a> |
                <a href="deleteProduit?id=${p.idProduit}" onclick="return confirm('Voulez-vous vraiment supprimer ce produit ?');">Supprimer</a>
            </td>
        </tr>
    </c:forEach>
</table>

</body>
</html>
```

FIGURE 16 – Page principale de l'application

4.7 Implémentation de l'authentification

Afin de sécuriser l'accès à l'application, un mécanisme d'authentification a été mis en place. L'utilisateur doit obligatoirement s'authentifier avant d'accéder aux différentes fonctionnalités du système.

Le processus repose sur la vérification des identifiants (nom d'utilisateur et mot de passe) via une Servlet dédiée appelée *LoginServlet*. En cas de succès, une session HTTP est créée afin de maintenir l'état de l'utilisateur durant sa navigation.

```java
public class LoginServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        if(username.equals("admin") && password.equals("123")) {

            HttpSession session = req.getSession();
            session.setAttribute("user", username);
            session.setAttribute("role", "ADMIN");

            resp.sendRedirect("listProduits");

        } else if(username.equals("user") && password.equals("123")) {

            HttpSession session = req.getSession();
            session.setAttribute("user", username);
            session.setAttribute("role", "USER");

            resp.sendRedirect("listProduits");

        } else {

            req.setAttribute("error", "Login ou mot de passe incorrect");
            req.getRequestDispatcher("login.jsp").forward(req, resp);
        }
    }
}
```

FIGURE 17 – Implémentation de LoginServlet

La gestion des utilisateurs repose sur une classe *User* contenant les informations nécessaires, notamment le nom d'utilisateur, le mot de passe et le rôle.

```java
public class User {

    private String username;
    private String password;
    private String role;

    public User() {}

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public String getRole() { return role; }

    public void setUsername(String username) { this.username = username; }
    public void setPassword(String password) { this.password = password; }
    public void setRole(String role) { this.role = role; }
}
```

FIGURE 18 – Classe User utilisée pour l'authentification

4.8 Implémentation des filtres (Filters)

Les filtres permettent d'intercepter les requêtes HTTP avant leur traitement par les Servlets. Ils jouent un rôle essentiel dans la sécurisation de l'application.

4.8.1 AuthFilter

Ce filtre vérifie si l'utilisateur est authentifié. Si aucune session valide n'est trouvée, l'utilisateur est redirigé vers la page de connexion.

```java
public class AuthFilter implements Filter {

    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);

        String uri = req.getRequestURI();

        if (uri.contains("login.jsp") || uri.contains("login")) {

            chain.doFilter(request, response);
            return;

        }

        if (session != null && session.getAttribute("user") != null) {

            chain.doFilter(request, response);

        } else {

            resp.sendRedirect("login.jsp");

        }
    }
}
```

FIGURE 19 – AuthFilter assurant la vérification de l'authentification

4.9 Configuration du fichier web.xml

Le fichier *web.xml* constitue un élément central de la configuration de l'application. Il permet de définir les servlets, les filtres ainsi que la gestion des erreurs.

```
<servlet>
    <servlet-name>login</servlet-name>
    <servlet-class>web.LoginServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>login</servlet-name>
    <url-pattern>/login</url-pattern>
</servlet-mapping>
```

FIGURE 20 – Configuration de LoginServlet dans le fichier web.xml

```
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>filters.AuthFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

FIGURE 21 – Configuration des filtres dans web.xml

```
<error-page>
    <error-code>404</error-code>
    <location>/error.jsp</location>
</error-page>
    <error-page> <error-code>500</error-code>
    <location>/error.jsp</location>
</error-page>
```

FIGURE 22 – Configuration de la gestion des erreurs dans le fichier web.xml

Dans ce fichier, les éléments suivants sont configurés :

— Le mapping des servlets (ListProduitServlet, AddProduitServlet, etc.)

— L'association des filtres aux URL protégées

— La gestion des erreurs (404, 500)

4.10 Gestion des erreurs

Une page dédiée *error.jsp* a été mise en place afin de gérer les erreurs de manière centralisée et améliorer l'expérience utilisateur.

Cette page est affichée en cas d'erreurs telles que :

— Ressource non trouvée (Erreur 404)

— Erreur interne du serveur (Erreur 500)

— Accès non autorisé

Cette approche permet d'éviter l'affichage d'erreurs techniques brutes et d'offrir une interface plus conviviale à l'utilisateur.

5 Mécanisme d'authentification

L'authentification constitue une étape essentielle garantissant que seuls les utilisateurs autorisés peuvent accéder à l'application.

Le processus d'authentification se déroule comme suit :

1. L'utilisateur saisit ses identifiants dans une interface dédiée (*login.jsp*)
2. Les données sont envoyées au serveur via une requête HTTP
3. Une Servlet de type *LoginServlet* vérifie la validité des informations
4. En cas de succès, une session HTTP est créée
5. L'utilisateur est redirigé vers l'interface principale

La session permet de conserver l'état de l'utilisateur tout au long de sa navigation.

6 Gestion des rôles

Afin de contrôler l'accès aux différentes fonctionnalités, un mécanisme de gestion des rôles a été mis en place.

Chaque utilisateur est associé à un rôle spécifique :

— **ADMIN** : dispose de tous les droits (ajout, modification, suppression)

— **USER** : accès restreint (consultation uniquement)

Cette approche permet d'implémenter une politique de sécurité basée sur le principe du moindre privilège.

7 Utilisation des Filters

Les filtres constituent un élément clé dans la gestion des requêtes HTTP. Ils permettent d'intercepter les requêtes avant leur traitement par les Servlets.

Deux filtres principaux ont été implémentés :

7.1 AuthFilter

Ce filtre vérifie si un utilisateur est authentifié. En absence de session valide, l'accès est refusé et une redirection vers la page de connexion est effectuée.

7.2 RoleFilter

Ce filtre contrôle les autorisations en fonction du rôle de l'utilisateur. Il empêche l'accès aux ressources sensibles pour les utilisateurs non autorisés.

8 Gestion des erreurs

Une gestion centralisée des erreurs a été mise en place afin d'améliorer l'expérience utilisateur et la robustesse du système.

Les erreurs suivantes sont prises en compte :

— Erreur 404 : ressource non trouvée

— Erreur 500 : erreur interne du serveur

— Accès interdit : tentative d'accès non autorisé

Ces erreurs sont redirigées vers une page dédiée (*error.jsp*) affichant un message explicite.

9 Fonctionnement global

Le fonctionnement global de l'application peut être résumé comme suit :

1. L'utilisateur s'authentifie
2. Les filtres vérifient l'accès
3. Les Servlets traitent les requêtes
4. La couche Service applique la logique métier
5. Le DAO interagit avec les données
6. Les résultats sont affichés via JSP

10 Conclusion

Ce projet a permis de mettre en œuvre une application web complète en Java EE en respectant les principes fondamentaux de l'architecture MVC1. L'intégration de mécanismes avancés tels que l'authentification, la gestion des rôles et l'utilisation des filtres a considérablement amélioré la sécurité et la qualité du système.

Ce travail constitue une base solide pour l'évolution vers des architectures plus avancées telles que MVC2 avec Spring Framework.

11 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées :

— Migration vers Spring Boot (architecture MVC2)
— Intégration d'une base de données relationnelle (MySQL)
— Mise en place d'une API REST
— Utilisation de JWT pour l'authentification
— Amélioration de l'interface utilisateur avec Bootstrap